AZURE DEVOPS

Complete Beginner's Guide

with Practical Project: Online Banking Application

Covers: Boards • Repos • Pipelines • Test Plans • Artifacts

Version 1.0 | 2026

Table of Contents

Table of Contents 2
1. What is Azure DevOps? 4
1.1 Five Core Services 4
1.2 Organization and Projects 4
1.3 Process Templates 5
2. Azure Boards — Plan and Track Work 6
2.1 Work Item Types 6
2.2 Practical Project — Online Banking App 6
2.3 How to Create Work Items — Step by Step 7
3. Boards — The Kanban View 9
3.1 Default Columns (Scrum Template) 9
3.2 Swimlanes 9
3.3 WIP Limits (Work In Progress) 9
4. Backlogs — The Prioritized Work List 11
4.1 Three Backlog Levels 11
4.2 Product Backlog vs Sprint Backlog 11
4.3 Backlog Grooming 11
5. Sprints — Time-Boxed Work Cycles 13
5.1 The Scrum Sprint Cycle 13
5.2 Setting Up Sprints in Azure DevOps 13
5.3 Sprint Capacity Planning 13
6. Queries — Search and Filter Work Items 14
6.1 Three Query Types 14
6.2 Building a Query — Fields, Operators, Values 14
6.3 My Queries vs Shared Queries 14
6.4 Practical Query Examples 15
7. Azure Repos — Source Code Management 16
7.1 Version Control Types 16
7.2 Key Git Concepts 16
7.3 Practical Workflow — Online Banking App 16
8. Azure Pipelines — Automate Build, Test & Deploy 18
8.1 CI vs CD — What's the Difference? 18
8.2 Pipeline Components 18
8.3 Practical Pipeline — Online Banking App 19

9. Azure Test Plans — Quality Assurance 20
9.1 Key Concepts 20
9.2 Practical Test Case — Online Banking App 20
10. Azure Artifacts — Package Management 21
11. Complete Project Walkthrough — Online Banking App 22
11.1 Project Setup (Day 1) 22
11.2 Sprint 1 Planning 22
11.3 Sprint 1 Execution 23
12. Tips for Beginners 24

1. What is Azure DevOps?

Azure DevOps is Microsoft's integrated, cloud-based platform that provides a complete set of tools to manage the entire software development lifecycle (SDLC). Think of it as an all-in-one toolkit where teams plan work, write code, build applications, test them, and deploy them to production — all in one place.

Simple Definition

Azure DevOps = Plan + Code + Build + Test + Deploy, all connected together. Instead of using 5 different tools for each step, you use one platform.

1.1 Five Core Services

Azure DevOps is made up of five integrated services, each covering a different part of the development process:

Service	Icon	What it Does	Used By
Azure Boards	Boards	Plan, track and manage work items — Epics, Features, Stories, Bugs, Tasks	Product Owners, Developers, QA
Azure Repos	Repos	Host and manage source code using Git or TFVC version control	Developers
Azure Pipelines	Pipelines	Automate building, testing, and deploying code (CI/CD)	DevOps Engineers, Developers
Azure Test Plans	Test	Create and run test cases, track test results and quality	QA Engineers, Testers
Azure Artifacts	Artifacts	Share and reuse code packages (NuGet, npm, Maven, etc.)	Developers, Architects

1.2 Organization and Projects

Azure DevOps is organized in two levels:

- Organization: The top-level container — like a company account. Example: 'OnlineBankingCorp'. You can have multiple projects inside one organization.
- Project: A workspace for a specific product or team. Example: 'OnlineBankingApp'. All the boards, repos, pipelines, and test plans for that product live here.

Access Levels

There are two main license types:

- Basic User: Full access to all Azure DevOps features. Used by developers, QA, PMs.
- Stakeholder: Free, limited access. Can view and create work items, but cannot use pipelines or advanced features.

1.3 Process Templates

When you create a project, you must choose a process template. This determines what work item types are available. Azure DevOps supports four templates:

Template	Best For	Backlog Items Called	Issues Called
Scrum	Agile/Scrum teams, sprints	Product Backlog Items (PBIs)	Impediments
Agile	Flexible agile teams	User Stories	Issues & Bugs
Basic	Simple projects, beginners	Issues	Issues
CMMI	Formal, regulated projects	Requirements	Issues, Change Requests

Practical Project Note

Throughout this guide we use the SCRUM template for our Online Banking App project. This means our backlog uses 'Product Backlog Items (PBIs)' and the hierarchy is: Epic > Feature > PBI > Task.

2. Azure Boards — Plan and Track Work

Azure Boards is the work management heart of Azure DevOps. It gives your team a digital space to create, assign, prioritize, and track all the work needed to build a product. It replaces sticky notes, spreadsheets, and email threads with a structured, visual system.

2.1 Work Item Types

A Work Item is any unit of work in the project. Each item has a title, description, assignee, state, and many other fields. In the Scrum template, these are the six work item types:

EPIC	The largest unit of work. Represents a major business goal or initiative that takes months and spans multiple teams. Too big for a single sprint — it breaks into Features. *Example: Build a complete Online Banking Platform*
FEATURE	A functional capability that delivers value to users. Lives inside an Epic and typically takes 2-4 weeks (several sprints). Breaks into PBIs. *Example: Fund Transfer Module, Account Statement View*
PRODUCT BACKLOG ITEM (PBI)	A specific user requirement written from the end-user perspective. Fits into one sprint (days). Format: 'As a [user], I want [action], so that [benefit].' *Example: Transfer money to another bank account*
TASK	The smallest unit of technical work. Hours (not days). Lives inside a PBI or Bug. Represents actual implementation steps. *Example: Build transfer API endpoint, Write unit tests*
BUG	A defect — something built but working incorrectly. Has Priority (1-4) and Severity fields. Goes through the same sprint workflow as PBIs. *Example: Transfer fails for amounts over Rs 10,000*
TEST CASE	Written by QA to verify PBIs work correctly. Contains step-by-step actions and expected results. Linked to PBIs. *Example: Verify successful transfer with valid amount*

2.2 Practical Project — Online Banking App

Let's see how the full hierarchy looks for our real project. Every item below is a real example you would create in Azure DevOps:

Complete Work Item Hierarchy

EPIC: Online Banking Application
- **FEATURE: Fund Transfer Module**
  - PBI #1042: Transfer money to another account (8 pts)
    - Task: Build POST /api/transfer endpoint
    - Task: Build transfer form UI
    - Task: Write unit tests
  - PBI #1043: Schedule future-dated transfer (5 pts)
  - BUG #2011: Transfer fails for amounts over Rs 10k [Priority 1]
- **FEATURE: Account Statement & History**
  - PBI #1044: View last 30 days transaction history (3 pts)
  - PBI #1045: Download monthly PDF statement (2 pts)
- **FEATURE: Login & Authentication**
  - PBI #1041: Login with email & password [Done]

2.3 How to Create Work Items — Step by Step

Step 1

Navigate to Boards

In your Azure DevOps project, click 'Boards' in the left sidebar, then click 'Backlogs'. You will see the backlog view.

Step 2

Create an Epic

Click 'New Work Item' at the top. Select 'Epic'. Enter Title: 'Online Banking Application'. Fill in Description: 'Build a complete online banking platform'. Click Save.

Step 3

Create a Feature under the Epic

From the backlog, expand the Epic. Click the '+' child button. Select 'Feature'. Enter Title: 'Fund Transfer Module'. This automatically links it to the Epic as a child.

Step 4

Create Product Backlog Items

Expand the Feature. Click '+' to add child. Select 'Product Backlog Item'. Enter the user story: 'As a customer, I want to transfer money to another account so I can pay people easily'. Set Story Points: 8. Set Priority: 2.

Step 5

Add Tasks to the PBI

Open the PBI. Scroll to 'Related Work' section. Click 'Add link' > 'New item'. Select 'Task'. Add task: 'Build POST /api/transfer endpoint'. Set Remaining Work: 4 hours. Assign to developer.

Step 6

Create a Bug

Click 'New Work Item' > 'Bug'. Title: 'Transfer fails for amounts over Rs 10,000'. Describe Steps to Reproduce, Expected Result, and Actual Result. Set Priority: 1 (Critical). Assign to developer.

3. Boards — The Kanban View

The Board is a visual Kanban-style view of your sprint's work items organized in columns. Each column represents a state in the workflow. You drag cards left to right as work progresses. This is used during daily standups to see what everyone is working on.

3.1 Default Columns (Scrum Template)

Column	Meaning
New	Work item has been created but no one has started it yet
Active	Someone is currently working on this item right now
Resolved	Developer says work is done, waiting for QA to verify
Closed	QA confirmed it is done and it is fully complete

3.2 Swimlanes

Swimlanes are horizontal rows on the board that categorize work differently. They answer the question: 'How should work be organized?'

- Default swimlane: All normal work items
- Expedite swimlane: High-priority or urgent items that need immediate attention
- Custom swimlanes: You can create your own — for example, one row per team or one row per Epic

To add a swimlane: Go to Board Settings > Swimlanes > Add Swimlane. You can customize color and name.

3.3 WIP Limits (Work In Progress)

WIP Limits prevent your team from taking on too much work at once. You can set a maximum number of cards allowed in any column. For example, setting the 'Active' column WIP limit to 3 means only 3 items can be in Active at any time. This forces the team to finish work before starting new items.

Board Customization Options

You can customize the board by going to Board Settings (gear icon):

- Add new columns (e.g. 'In Review' between Active and Resolved)
- Split a column into 'Doing' and 'Done' sub-columns
- Set WIP limits per column
- Add style rules to color-code cards (e.g. all Priority 1 bugs show in red)
- Use # to search for users and @ to mention/assign users in comments

4. Backlogs — The Prioritized Work List

A Backlog is an ordered, prioritized list of all the work that needs to be done. It is the single source of truth for what the team needs to build. Items at the top have the highest priority and will be worked on first.

4.1 Three Backlog Levels

Backlog Level	Shows
Epics Backlog	All Epics — strategic view used by management and product managers. Shows big goals.
Features Backlog	All Features grouped under Epics — used by Product Owners for sprint-level planning.
Stories/PBI Backlog	All Product Backlog Items and Bugs — the main working backlog used by developers every sprint.

4.2 Product Backlog vs Sprint Backlog

	Product Backlog
What it contains	ALL future work — every PBI, Bug, Feature not yet in a sprint
Who manages	Product Owner
Duration	Ongoing — never fully empty
Can it change?	Yes, continuously refined and reprioritized
	Sprint Backlog
What it contains	Only items committed for the CURRENT sprint (2 weeks)
Who manages	Development Team
Duration	Fixed for 1-2 weeks (one sprint)
Can it change?	No — locked during the sprint after planning

4.3 Backlog Grooming

Backlog Grooming (also called Backlog Refinement) is a regular meeting where the Product Owner and team review the backlog together. During grooming you:

- Add story points to items that have none
- Split large PBIs into smaller, more manageable ones
- Remove outdated or irrelevant items
- Reprioritize items based on business needs
- Write or update Acceptance Criteria for upcoming items

To move items to a sprint: right-click any backlog item > 'Move to Sprint' > select the sprint. Or drag items from the backlog into the sprint panel on the right side of the screen.

5. Sprints — Time-Boxed Work Cycles

A Sprint (also called an Iteration) is a fixed time period — usually 2 weeks — during which the team commits to completing a specific set of work items. Sprints are the heartbeat of Scrum methodology.

5.1 The Scrum Sprint Cycle

Sprint Lifecycle

1. Sprint Planning: Team selects PBIs from the Product Backlog and commits to completing them in this sprint.
2. Sprint Backlog: Selected items move to the sprint backlog and are broken into tasks.
3. Sprint Execution: Team works for 2 weeks. Daily Standup each day (15 mins): What did I do? What will I do? Any blockers?
4. Sprint Review: Demo the completed work to stakeholders.
5. Sprint Retrospective: Team discusses what went well, what to improve.

5.2 Setting Up Sprints in Azure DevOps

Go to Project Settings > Boards > Team Configuration > Iterations. Here you can:

- Create new sprints (Sprint 1, Sprint 2, etc.)
- Set start and end dates for each sprint
- Configure working days (e.g. Monday to Friday)
- Set backlog navigation levels (Epics, Features, PBIs)

5.3 Sprint Capacity Planning

Before committing to sprint work, set capacity for each team member. Go to: Boards > Sprints > [Sprint Name] > Capacity tab.

- Set days per week each person works (e.g. 5 days)
- Set hours per day (e.g. 6 hours)
- Set any days off (holidays, vacation)
- Azure DevOps automatically calculates total team capacity

Use this to ensure you do not over-commit by adding more work than the team can realistically complete.

6. Queries — Search and Filter Work Items

A Query is a saved search that filters work items based on conditions you define. It's like a custom SQL WHERE clause for your project data. Once saved, a query runs live every time you open it — always showing fresh, up-to-date results.

6.1 Three Query Types

Query Type	Description & Use Case
Flat List	Simple table of work items matching your filters. Most common type. Use case: 'Show all active bugs assigned to me in the current sprint.'
Tree of Work Items	Shows parent-child hierarchy (Epic > Feature > PBI > Task). Use case: 'Show all work items under the Fund Transfer Feature.'
Work Items and Direct Links	Shows work items alongside related items (blocking, related, duplicate). Use case: 'What is currently blocking our sprint items?'

6.2 Building a Query — Fields, Operators, Values

Every query clause has three parts: Field + Operator + Value. Multiple clauses are joined with AND or OR.

Field	Operator	Value	Meaning
Work Item Type	= (equals)	Bug	Only show Bugs
State	<> (not equals)	Closed	Exclude closed items
Assigned To	= (equals)	@Me	Only my items
Iteration Path	= (equals)	@CurrentIteration	Current sprint only
Priority	<= (less than equal)	2	Priority 1 and 2 only
Title	Contains	payment	Any item with 'payment' in title
Changed Date	>= (greater equal)	@Today - 7	Changed in last 7 days

Special Variables — These Update Automatically

@Me — Automatically resolves to the currently logged-in user
@Today — Today's date. Use @Today - 7 for last week, @Today - 30 for last month
@CurrentIteration — Active sprint for your team. Never need to update manually.
@StartOfMonth — First day of the current calendar month

6.3 My Queries vs Shared Queries

- My Queries: Private queries visible only to you. Good for personal daily tracking.
- Shared Queries: Visible to the whole team. Organized in folders. Can be used as data sources for Dashboard Widgets.

6.4 Practical Query Examples

Query Name	Clause Logic
My active bugs this sprint	Type=Bug AND State=Active AND Assigned To=@Me AND Iteration=@CurrentIteration
All items in current sprint	Iteration=@CurrentIteration AND State<>Closed AND Type In (PBI, Bug, Task)
Critical unresolved bugs	Type=Bug AND Priority<=2 AND State<>Closed AND State<>Resolved
Unestimated backlog items	Type=PBI AND Story Points=[empty] AND State<>Closed
Items I changed this week	Changed By=@Me AND Changed Date>=@Today-7
Items completed this sprint	Iteration=@CurrentIteration AND State=Done AND Type In (PBI, Bug)

7. Azure Repos — Source Code Management

Azure Repos is the version control service in Azure DevOps. It stores your code safely, tracks every change ever made, enables multiple developers to work on the same codebase simultaneously, and prevents conflicts through branching and merging.

7.1 Version Control Types

Type	Description
Git (Default)	Distributed Version Control System (DVCS). Every developer has a full copy of the repository. Industry standard. Most popular choice.
TFVC (Team Foundation)	Centralized Version Control System (CVCS). Single repository on the server. Older approach, still supported for legacy projects.

7.2 Key Git Concepts

- Repository (Repo): The storage location for all your project's code and history
- Branch: An independent line of development. The main branch (called 'main' or 'master') is production code. Feature branches are for new work.
- Commit: Saving a snapshot of your changes with a message describing what you did
- Push: Uploading your local commits to the remote Azure Repos server
- Pull Request (PR): Requesting that your branch be merged into main. Enables code review before merging.
- Merge: Combining code from one branch into another

7.3 Practical Workflow — Online Banking App

Step 1 — **Clone the repository**

git clone https://dev.azure.com/OnlineBankingCorp/OnlineBankingApp/_git/BankingRepo

Step 2 — **Create a feature branch**

git checkout -b feature/fund-transfer — Creates a new branch for your work without touching the main branch

Step 3 — **Write your code**

Develop the fund transfer API endpoint. Stage and commit your work: git add . then git commit -m 'Add POST /api/transfer endpoint'

Step 4 — **Push your branch**

git push origin feature/fund-transfer — Uploads your branch to Azure Repos

Step 5

Create a Pull Request

In Azure DevOps, go to Repos > Pull Requests > New PR. Select your branch, add a description, assign reviewers (e.g. senior developer). This triggers code review.

Step 6

Code review and merge

Reviewers comment on the code, request changes, or approve. Once approved, merge the PR into main. The feature branch is deleted.

8. Azure Pipelines — Automate Build, Test & Deploy

Azure Pipelines is the CI/CD (Continuous Integration / Continuous Deployment) service. It automatically builds, tests, and deploys your code whenever a developer pushes changes. This eliminates manual build steps and catches bugs early.

8.1 CI vs CD — What's the Difference?

	Continuous Integration (CI)
What it does	Automatically builds and runs tests every time code is pushed or merged
Goal	Catch bugs early, ensure code always compiles and passes tests
Trigger	Every code commit/merge to the main branch
Output	Build artifacts (compiled code ready to deploy)
	Continuous Deployment (CD)
What it does	Automatically deploys built artifacts to environments (Dev, QA, Production)
Goal	Get working software into the right environment reliably and quickly
Trigger	Successful CI build or manual approval
Output	Running application in target environment

8.2 Pipeline Components

A pipeline is defined in a YAML file stored alongside your code. It has these nested components:

Component	Description
Trigger	The event that starts the pipeline. Example: push to main branch, pull request created, nightly schedule
Stage	A logical grouping of jobs. Represents a deployment environment: Build stage, Test stage, Deploy stage.
Job	A single phase within a stage. Jobs run on an Agent. Multiple jobs in a stage can run in parallel.
Step	The smallest unit. A Step is one action inside a job: run a script, execute a task, publish an artifact.
Task	A pre-built script provided by Microsoft or the marketplace. Example: 'DotNetCoreCLI@2' to build .NET apps.
Agent	The computer that executes the pipeline. Microsoft-hosted (ready-made VM) or Self-hosted (your own server).

Artifact	The output of the build stage — compiled code, packages, Docker images — passed to the deploy stage.
Approval	A manual gate requiring a human to approve before the pipeline continues to the next stage.

8.3 Practical Pipeline — Online Banking App

Here is a real YAML pipeline for the Online Banking App. Save this file as 'azure-pipelines.yml' in your repository root:

```yaml
# azure-pipelines.yml
trigger:
  - main

stages:
- stage: Build
  jobs:
  - job: BuildJob
    pool:
      vmImage: 'ubuntu-latest'  # Microsoft-hosted agent
    steps:
    - task: DotNetCoreCLI@2
      inputs:
        command: 'build'
    - task: DotNetCoreCLI@2
      inputs:
        command: 'test'   # Run unit tests
- stage: Deploy
  dependsOn: Build      # Only runs if Build succeeds
  jobs:
  - deployment: DeployToAzure
    environment: 'Production'
```

9. Azure Test Plans — Quality Assurance

Azure Test Plans provides a complete manual and automated testing solution. QA engineers use it to create test cases, organize them into test suites, execute them against each sprint's work, and track pass/fail results over time.

9.1 Key Concepts

Concept	Description
Test Plan	A container for all test activities for one sprint or release. Example: 'Online Banking Sprint 3 Test Plan'
Test Suite	A collection of test cases grouped by feature or PBI. Example: 'Fund Transfer Tests', 'Login Tests'
Test Case	Individual test with step-by-step actions and expected results. Each step has an Action and Expected Result.
Test Run	An execution session where a tester runs through selected test cases and records Pass/Fail for each step
Defect (Bug)	When a step fails, a Bug is automatically created and linked to the test case and the PBI being tested

9.2 Practical Test Case — Online Banking App

Test Case #3001: Verify successful transfer with valid amount

Linked to: PBI #1042 — Transfer money to another account

Step 1 Action: Log in with valid credentials | Expected: Dashboard is displayed
Step 2 Action: Click 'Transfer Money' | Expected: Transfer form opens successfully
Step 3 Action: Enter recipient account 9876543210, amount Rs 500 | Expected: Fields accept valid input, no error
Step 4 Action: Click 'Review & Confirm' | Expected: Confirmation screen shows correct details
Step 5 Action: Click 'Confirm Transfer' | Expected: Success message and reference number displayed

10. Azure Artifacts — Package Management

Azure Artifacts lets teams create, host, and share packages (reusable code libraries) within the organization. Instead of every project re-writing the same utility code, you publish it as a package once and all projects reference it.

Package Type	Used For
NuGet	.NET / C# libraries and packages
npm	JavaScript / Node.js packages
Maven	Java packages
PyPI	Python packages
Universal Packages	Any file type — binaries, scripts, archives

In the Online Banking App, you might create an Artifact called 'BankingUtils' containing shared functions for account validation, currency formatting, and authentication. Both the frontend app and backend API can reference this shared package.

11. Complete Project Walkthrough — Online Banking App

This section walks through the complete lifecycle of the Online Banking App project from Day 1 to Sprint 3, showing exactly how every Azure DevOps feature is used in a real scenario.

11.1 Project Setup (Day 1)

1 **Create Organization**
Go to dev.azure.com. Click 'New Organization'. Name: 'OnlineBankingCorp'. This is the company-level container.

2 **Create Project**
Inside the organization, click 'New Project'. Name: 'OnlineBankingApp'. Process Template: Scrum. Visibility: Private. Version Control: Git.

3 **Add Team Members**
Go to Project Settings > Teams > select your team > Add members. Set access levels: Developers = Basic User. Managers = Stakeholder.

4 **Configure Sprints**
Go to Project Settings > Boards > Team Configuration. Create sprints: Sprint 1 (Jan 1-14), Sprint 2 (Jan 15-28), Sprint 3 (Jan 29 - Feb 11). Set working days: Mon-Fri.

11.2 Sprint 1 Planning

5 **Create the Epic**
Navigate to Boards > Backlogs. Create Epic: 'Online Banking Application'. Description: Build a complete online banking platform.

6 **Create Features**
Under the Epic create: 'Login & Authentication', 'Fund Transfer Module', 'Account Statement View', 'Bill Payment Module'.

7 **Fill Sprint 1 Backlog**
Under 'Login & Authentication' Feature, create PBIs: (1) Login with email & password [5 pts], (2) Show account balance on dashboard [3 pts], (3) Logout functionality [1 pt].

8 **Set Capacity**

Go to Boards > Sprints > Sprint 1 > Capacity. Set Ravi Kumar: 6 hrs/day, 5 days. Anjali Singh: 6 hrs/day, 5 days. Total: ~60 hours of capacity.

9 **Move items to Sprint 1**
Right-click each PBI in the backlog and select 'Move to Sprint 1'. Total: 9 story points, within team capacity.

11.3 Sprint 1 Execution

10 **Daily Standup with Board**
Each morning open Boards > Boards > Sprint 1. Each developer drags their task cards from New > Active as they start work, and to Done when complete.

11 **Bug found during sprint**
QA discovers Login fails with special characters in password. Create Bug: 'Bug #2001: Login fails for passwords with special characters.' Priority 1. Assign to Ravi Kumar.

12 **Run a Query**
QA creates Shared Query: 'Sprint 1 Active Bugs'. Clauses: Type=Bug AND State=Active AND Iteration=@CurrentIteration. Saves in Shared Queries > Sprint 1 folder.

13 **Sprint 1 Retrospective**
Using Boards > Sprints > Sprint 1 analytics, team reviews velocity (9/9 points = 100% complete). Add retrospective notes in a Markdown dashboard widget.

Sprint 2 and 3 follow the same cycle — plan, execute, review. By Sprint 3 the Fund Transfer Module is complete, with all PBIs done and critical bugs resolved.

12. Tips for Beginners

Top 10 Things Every Beginner Should Know

1. Always use the Scrum template when starting — it is the most common in industry.
2. Work Item hierarchy: Epic > Feature > PBI > Task. Never create Tasks without a parent PBI.
3. Story Points estimate effort, not time. Use Fibonacci: 1, 2, 3, 5, 8, 13, 21.
4. Use @Me and @CurrentIteration in queries — they auto-update and never go stale.
5. Shared Queries are the backbone of Dashboard Widgets. Always organize them in folders.
6. Every Bug needs: Steps to Reproduce, Expected Result, Actual Result, Environment. Without this, developers cannot fix it.
7. Never commit code directly to 'main'. Always create a feature branch and use Pull Requests.
8. Pipelines fail for a reason. Always check the failed step logs — they tell you exactly what went wrong.
9. Groom your backlog every sprint. Unestimated items block planning. Use query 'Story Points = [empty]' to find them.
10. Use the Azure DevOps Demo Generator at https://azuredevopsdemogenerator.azurewebsites.net to create a free sample project for practice.

— End of Azure DevOps Beginner Guide —